



UNIVERSITÀ DEGLI STUDI
DI SALERNO

Corso di Fondamenti di Intelligenza Artificiale

RICONOSCIMENTO AUTOMATICO DELLA LIS



Repository Github:

https://github.com/Swami-Falasca/LIS_AI.git

Dima Isabella, Falasca Swami, Tirelli Giuseppe

Indice

1	Introduzione	1
2	Descrizione dell'agente	1
2.1	Obiettivi	1
2.2	Specifica PEAS	2
2.2.1	Caratteristiche dell'ambiente	2
2.3	Analisi del Problema	2
2.3.1	Formalizzazione del problema	3
3	Raccolta e preprocessing dei dati	3
3.1	Scelta del dataset	3
3.2	Analisi del dataset	4
3.3	Operazioni di preprocessing	4
3.3.1	Riduzione delle dimensioni del problema	4
3.3.2	Tecniche di smoothing e normalizzazione	5
3.3.3	Data argumentation (flipping)	5
3.4	Librerie	5
4	Glossario	5

1 Introduzione

La Lingua dei Segni Italiana (LIS) è una lingua visivo-gestuale e rappresenta un linguaggio vero e proprio, con una propria grammatica.

Il progetto **LIS_AI** propone un sistema di riconoscimento automatico della LIS per contrastare questa disinformazione della popolazione italiana. E' basato su, per tradurre gesti in tempo reale. L'obiettivo è facilitare la comunicazione tra sordi e udenti senza richiedere conoscenze della LIS

2 Descrizione dell'agente

2.1 Obiettivi

Lo scopo di questo progetto è realizzare un agente intelligente in grado di riconoscere gesti della Lingua dei Segni Italiana (LIS) da immagini e tradurli in testo. L'agente deve essere in grado di:

1. Riconoscere i gesti LIS, fondamentale per comunicare in modo efficace con le persone sorde
2. Tradurre i gesti riconosciuti in testo in tempo reale
3. Mettere a disposizione un'interfaccia semplice per l'utilizzo del sistema

2.2 Specifica PEAS

Performance	Accuratezza: Corretta classificazione dei gesti LIS Tempo di risposta: Traduzione in tempo reale Robustezza: Diverse condizioni di illuminazione
Environment	Input: Video acquisiti via webcam Posizione: Utente e sfondo variabile Illuminazione: Non controllata Distrazioni: Potenziale presenza di elementi distraenti
Actuators	Output: Visualizzazione della lettera tradotta Interfaccia: Comandi avvio/arresto, salvataggio, pulizia cronologia Feedback: Indicazione visiva della confidenza
Sensors	Webcam: Acquisizione video Frame: Elaborazione dei frame Features: Estrazione delle caratteristiche visive

2.2.1 Caratteristiche dell'ambiente

- **Parzialmente osservabile:** non tutti i dettagli dei gesti sono sempre visibili
- **Dinamico:** l'utente si muove liberamente durante l'esecuzione dei gesti
- **Sequenziale:** i gesti hanno una dimensione temporale da considerare
- **Discreto:** insieme finito di gesti da riconoscere

2.3 Analisi del Problema

La sfida del riconoscimento della Lingua Italiana dei Segni è stata affrontata come un problema di classificazione. I gesti sono stati infatti suddivisi ed etichettati per lettera rappresentata. Tuttavia, nella progettazione sono sorti numerosi problemi di grande importanza per la buona riuscita del progetto:

- I segni LIS sono veicolati con la mano dominante. La mano utilizzata deve essere quindi influente nel riconoscimento del segno.

- Come rappresentare al meglio i dati in modo da estrarre solo le informazioni essenziali?
- Come scegliere un formato adatto, ponderando anche l'alleggerimento del carico computazionale e l'ottimizzazione della performance?
- Non tutte le lettere della LIS sono riconosciute allo stesso modo. Ad esempio, la lettera "S" richiede un movimento della mano. Potrebbe essere opportuno utilizzare più di un modello di classificazione.

2.3.1 Formalizzazione del problema

- **Stato iniziale:** Frame video non processato dalla webcam
- **Azioni possibili:**
 - Acquisire nuovo frame dalla webcam
 - Classificare il gesto utilizzando il modello addestrato
 - Visualizzare il risultato della classificazione
- **Test obiettivo:** Riconoscimento corretto del gesto eseguito
- **Costo del cammino:** Errore di classificazione e tempo di computazione

La complessità del problema richiede l'esplorazione di multiple architetture neurali per identificare il miglior compromesso tra accuratezza e efficienza computazionale.

Il miglior compromesso tra accuratezza ed efficienza è fondamentale. Per trovare ciò ci dobbiamo basare sulla velocità con cui otteniamo i risultati e l'accuratezza dei risultati ottenuti.

3 Raccolta e preprocessing dei dati

3.1 Scelta del dataset

Il dataset scelto per questo progetto è il **LIS Dataset** disponibile su Kaggle, contenente oltre 6000 immagini in formato JPG delle lettere dell'alfabeto italiano della Lingua dei Segni Italiana opportunamente etichettate. Sono presenti tutte le lettere dell'alfabeto italiano ad eccezione delle seguenti: G, J, S e Z perchè richiedono movimenti più complessi.

La scelta è ricaduta su questo dataset per vari motivi:

- Rappresenta solo la LIS e non l'ASL
- È di dimensioni gestibili per le risorse computazionali disponibili

Nonostante l'ottimo dataset, non possiamo ignorare il fatto che è, purtroppo, **incompleto**. Non essendoci su Kaggle o piattaforme simili alternative valide, abbiamo optato per registrare noi stessi le lettere rimanenti (record_dynamic.py), ottenendo così 100 campioni di ogni lettera. Le modalità di registrazione delle lettere mancanti verranno approfondite nella sezione (...)

Per maggiore chiarezza, da adesso in poi ci riferiremo a tutte le lettere presenti nel dataset originale, ovvero rappresentabili con una semplice immagine statica, come **lettere statiche**. Le lettere non rappresentabili staticamente, ovvero richiedenti un movimento, saranno definite **lettere dinamiche**.

3.2 Analisi del dataset

Il dataset originale presenta le seguenti caratteristiche:

- Numero di classi: 6428 foto LIS
- Formato: foto in JPG
- Risoluzione: variabile tra le foto
- Sfondo: non uniforme tra i diversi campioni

3.3 Operazioni di preprocessing

Per preparare i dati all'addestramento dei modelli, sono state effettuate le seguenti operazioni:

3.3.1 Riduzione delle dimensioni del problema

Per gestire al meglio le informazioni del dataset, eliminando i dettagli non essenziali e focalizzando l'attenzione sui contenuti rilevanti abbiamo optato per una riduzione delle dimensioni dei dati.

In particolare, sfruttando la libreria `mediapipe`, abbiamo identificato sulle mani 21 landmarks (punti chiave anatomici), corrispondenti a polso (1 punto) e giunture delle dita (4 punti per dito = 20 punti). I landmarks connessi tra loro che sono stati rappresentati come vettori di coordinate (x,y) , per un totale di 42 features per immagine.

La struttura dati contenente l'insieme di tutti i vettori di landmarks etichettati è stata serializzata in formato `pickle`, particolarmente adatto al nostro problema per il suo supporto nativo per array `NumPy` e gli oggetti `scikit-learn`.

Questo ci consente di lavorare con un dataset ridotto alle informazioni essenziali, riducendo drasticamente il carico computazionale. Da diverse centinaia di MegaByte di immagini `jpg` si passa ad un singolo file di meno di 10 MegaByte.

Le registrazioni delle lettere dinamiche (J, Z, S, G) abbiamo adottato una strategia analoga. Per semplicità, abbiamo saltato il passaggio da formato video a immagini statiche e registrato direttamente i vettori di landmarks per ogni frame dei 30 totali per ogni acquisizione.

Le registrazioni sono state effettuate eseguendo il file `record_dynamic.py`, anch'esso disponibile nel repository GitHub del progetto. In totale, sono state acquisite 100 registrazioni per ogni lettera dinamica

3.3.2 Tecniche di smoothing e normalizzazione

Per quanto riguarda le lettere statiche abbiamo implementato una normalizzazione spaziale. Per rendere i gesti invarianti alla posizione della mano nell'immagine, abbiamo centrato le coordinate dei landmarks rispetto al polso.

Per le lettere dinamiche, essendo composte da una sequenza di frame, abbiamo dovuto seguire un approccio diverso. I gesti sono stati infatti normalizzati e scalati rispetto ai minimi e ai massimi tenendo conto di tutta la traiettoria, e non solo di un singolo frame. Abbiamo poi applicato dei filtri per ridurre il rumore all'interno del dataset, filtrando le fluttuazioni rapide ed eliminando così gli outlier dovuti a registrazioni imperfette.

3.3.3 Data argumentation (flipping)

Come spiegato nella sezione 2.3, la mano utilizzata nei segni deve essere ininfluente per il riconoscimento.

Per evitare bias di lateralità (ovvero la tendenza del modello a riconoscere più facilmente gesti di una mano piuttosto che dell'altra) e aumentare la robustezza del modello, abbiamo effettuato un'operazione di data augmentation tramite "flipping". Questa tecnica ci permette di simulare la mano opposta senza l'inserimento di dati aggiuntivi.

Nel dettaglio, dopo aver estratto i landmarks in ogni immagine, abbiamo invertito le coordinate orizzontali (intuitivamente abbiamo, quindi, capovolto le mani) e aggiunto i vettori "flipped" al dataset di landmarks.

Così facendo, il dataset finale è raddoppiato e il modello finale sarà invariante alla lateralità, simulando così con maggiore efficacia la simmetria intrinseca di molti gesti della LIS.

3.4 Librerie

Le librerie che sono state usate per sviluppare il sistema sono:

- ◇ mediapipe: serve ad elaborare immagini e video.
- ◇ matplotlib.pyplot: serve per creare visualizzazioni interattive, e per creare grafici.
- ◇ pickle: serializzazione strutture dati e modelli addestrati
- ◇ numpy: gestione di array

Per i modelli statici:

- ◇ Scikit-Learn: usata per i classificatori e le metriche

Per i modelli dinamici:

- ◇ keras: addestramento di reti neurali

4 Glossario